Advanced Data Structures

Lecture 3 – Amortized Analysis II: The Accounting Method

Dr. Suraj Kumar

Department of Computer Science & Engineering

31 July 2026

*Amortized analysis, part 2 of 3 (aggregate → **accounting** → potential)*

Where We Are

Recall: the aggregate method

Bound the total cost $T(n)$ of any n operations, then the amortized cost is $\hat{c} = T(n)/n$. We proved $O(1)$ amortized for the multipop stack, the binary counter, and the dynamic table.

Its one weakness

The aggregate method gives the *same* amortized cost to *every* operation. It cannot say "PUSH pre-pays for a later POP".

Three methods

1. Aggregate – Lecture 2
2. **Accounting** – *today*
3. Potential – Lectures 4–5

Today: charge each operation individually, and let cheap operations *pre-pay* for expensive ones.

The Accounting Method

Also called the banker's method

Definition (accounting method)

Assign to each operation an **amortized charge** $\hat{c}_i$, possibly different from its actual cost c_i. When $\hat{c}_i > c_i$, the surplus is stored as **credit** inside the data structure; when $\hat{c}_i < c_i$, the deficit is paid from credit stored earlier.

The golden rule

The total stored credit must never go negative:

$$\sum_{j=1}^{i} (\hat{c}_j - c_j) \;\geq\; 0 \quad \text{for every } i.$$

You may only spend credit you have already banked.

Why it works

If the golden rule holds, then for every n

$$\sum_{i=1}^{n} c_i \;\leq\; \sum_{i=1}^{n} \hat{c}_i,$$

so the (easy) total charge bounds the (real) total cost.

The advantage over aggregate: different operations may carry different charges. The art is choosing the charges and where to store the credit.

Example 1 – Stack with MULTIPOP

Choosing the charges

The charge scheme

Operation	**Actual** c_i	**Charge** $\hat{c}_i$
PUSH	1	2
POP	1	0
MULTIPOP	$\min(k, \lvert S \rvert)$	0

Where the credit lives

PUSH pays 1 to do the push and banks 1 **credit** *on the pushed element.*

So **every element on the stack carries exactly** 1 **credit** – its own future removal, pre-paid.

Notice the charges differ by operation – something the aggregate method could not express.

Example 1 – Why It Works

The credit is exactly the stack size

The invariant

Each element on the stack holds 1 credit, so

$$\text{total credit} \;=\; |S| \;\geq\; 0.$$

A Pop or Multipop removes elements at actual cost 1 each, paid *entirely* from the credit those elements carry – so their charge can be 0.

$$\sum \hat{c}_i = 2 \cdot (\#\textsc{push}) \leq 2n = O(n).$$

A concrete run ($n = 8$)

Operation	c_i	$\hat{c}_i$	**Credit**
Push a	1	2	1
Push b	1	2	2
Push c	1	2	3
Push d	1	2	4
Multipop(3)	3	0	1
Push e	1	2	2
Push f	1	2	3
Multipop(10)	3	0	0
Total	**12**	**12**	

Credit never negative; $\sum c_i \leq \sum \hat{c}_i$.

Example 2 – Binary Counter

One credit per 1-bit

The charge scheme

Charge every INCREMENT an amortized cost of $\hat{c} = 2$:

- 1 pays to set the single bit $0 \to 1$;
- 1 is banked as **credit** *on that newly-set* 1 *bit.*

Who pays for the resets?

An INCREMENT may reset many bits $1 \to 0$. Each such reset costs 1, paid by the credit sitting on that 1 bit. The charge only has to cover the single $0 \to 1$ set.

The invariant

Every 1 bit in the counter carries exactly 1 credit, so

$$\text{total credit} = (\#\,1\text{-bits}) \geq 0.$$

Hence

$$\sum \hat{c}_i = 2n = O(n), \qquad \hat{c} = O(1).$$

Example 2 – A Concrete Run

Credit always equals the number of 1-bits

After	Counter	c_i	$\hat{c}_i$	Credit
0	00000	–	–	0
1	00001	1	2	1
2	00010	2	2	1
3	00011	1	2	2
4	00100	3	2	1
5	00101	1	2	2
6	00110	2	2	2
7	00111	1	2	3
8	01000	4	2	1

The **credit** column is just the number of 1s in the counter – it dips when a costly INCREMENT resets a run of 1s, but it is never negative.

After $n = 8$

$$\sum c_i = 15 \leq \sum \hat{c}_i = 16.$$

Example 3 – Dynamic Table (Table Doubling)

Three coins per insertion

The charge scheme

Charge every INSERT an amortized cost of $\hat{c} = 3$:

- 1 pays to place the new item now;
- 2 are banked as **credit** *on that item.*

Who pays to copy at an expansion?

When a full table of m items doubles, all m items are copied (cost m). The $m/2$ items inserted since the previous expansion each carry 2 credits – together $2 \cdot \frac{m}{2} = m$ – exactly enough to move *everyone.*

The bound

Every insertion is charged 3, so

$$\sum \hat{c}_i = 3n = O(n), \qquad \hat{c} = O(1),$$

matching the $T(n) < 3n$ we found with the aggregate method

Example 3 – Following the Credit

Charge 3; credit dips at each doubling but stays ≥ 0

Insert i	c_i	$\hat{c}_i$	**Credit**
1	1	3	2
2	2	3	3
3	3	3	3
4	1	3	5
5	5	3	3
6	1	3	5
7	1	3	7
8	1	3	9

c_i includes copying at expansions ($i = 1, 2, 3, 5, \dots$).

At each expansion ($i = 5$ here) the credit is spent down to move the old items, but it never falls below 0.

After $n = 8$

$$\sum c_i = 15 \leq \sum \hat{c}_i = 24$$

and the leftover credit (9) is prepaid work for future expansions.

Aggregate vs. Accounting

What they share

- Both prove the same bounds: $O(1)$ amortized, $O(n)$ total.
- Both are worst-case guarantees – no probability.

The price you pay

- You must *invent* the charges and decide where credit is stored.
- You must prove the golden rule (credit ≥ 0) by hand.

Accounting adds

- *Per-operation* charges (PUSH $= 2$, POP $= 0$).
- A concrete story: credit is prepaid future work.

Coming next

The **potential method** (Lectures 4–5) makes the credit a single function Φ of the data structure's state – systematising exactly this bookkeeping.

Summary

Key points

- Accounting: charge $\hat{c}_i$ per operation; bank surplus as credit, spend it on expensive operations.
- **Golden rule:** credit $= \sum(\hat{c}_j - c_j) \geq 0$ always, hence $\sum c_i \leq \sum \hat{c}_i$.
- **Multipop stack:** PUSH $= 2$, POP/MULTIPOP $= 0$; 1 credit per stacked element.
- **Binary counter:** INCREMENT $= 2$; 1 credit per 1-bit.
- **Dynamic table:** INSERT $= 3$; 2 credits per item pay for future copying.

Next lecture (L4)

The **potential method**: define Φ so that $\hat{c}_i = c_i + \Phi_i - \Phi_{i-1}$, and re-derive all of today's results.

Reading

- CLRS, §17.2 (the accounting method)
- Compare with your §17.1 aggregate proofs

Exercises

Try before Lecture 4

1. Prove the golden rule formally: if $\sum_{j \le i} (\hat{c}_j - c_j) \ge 0$ for all i, then $\sum_{i \le n} c_i \le \sum_{i \le n} \hat{c}_i$ for all n.
2. For the dynamic table, show that a charge of 2 per insertion is *not* enough – exhibit a point where the credit would go negative – and confirm that 3 is the smallest integer charge that works.
3. Add $\textsc{Multipush}(S, x, k)$ (push k copies of x, cost k) to the stack. Give amortized charges for all four operations, or argue that no constant charge for $\textsc{Multipush}$ can work.
4. Suppose the counter also supports $\textsc{Decrement}$. Show that no fixed "1 credit per 1-bit" scheme keeps the amortized cost $O(1)$ when $\textsc{Increment}$ and $\textsc{Decrement}$ may alternate.